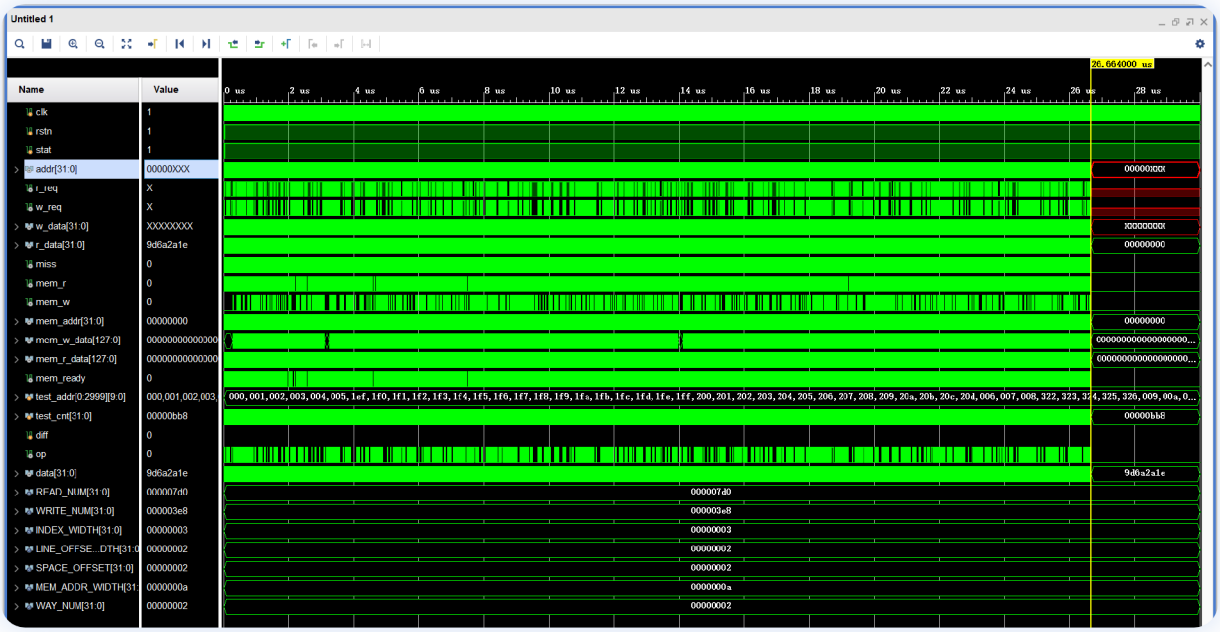
# Cache 性能分析报告

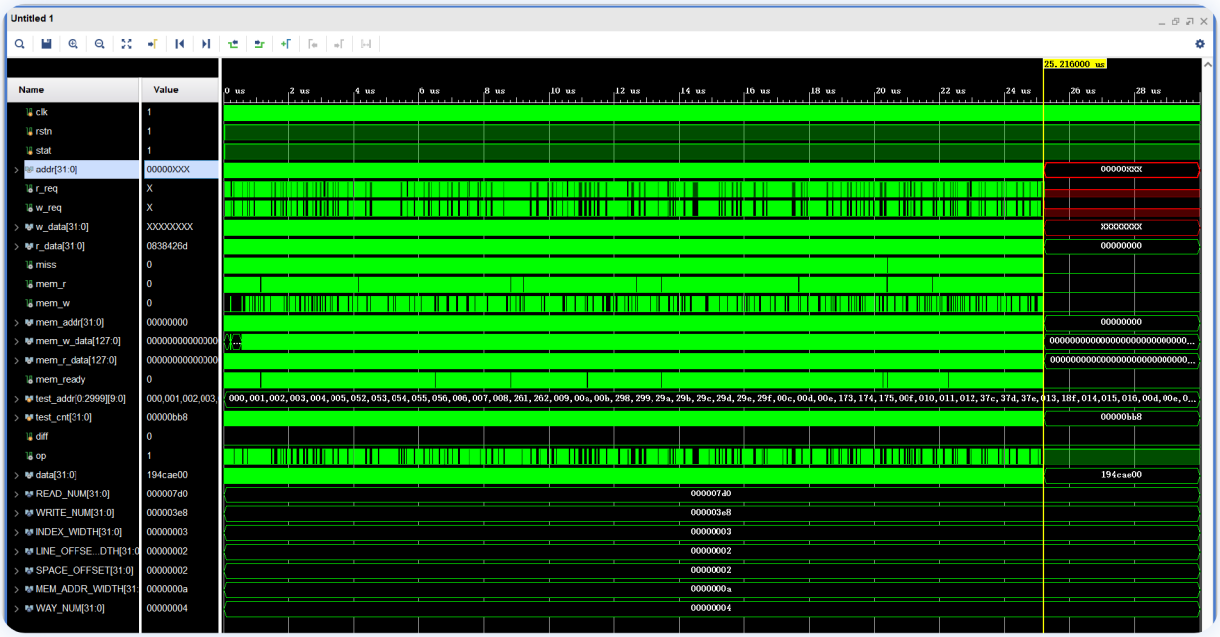
## 任务 1：N 路组相连 Cache

可见随着路数增加，累计用时有明显下降。

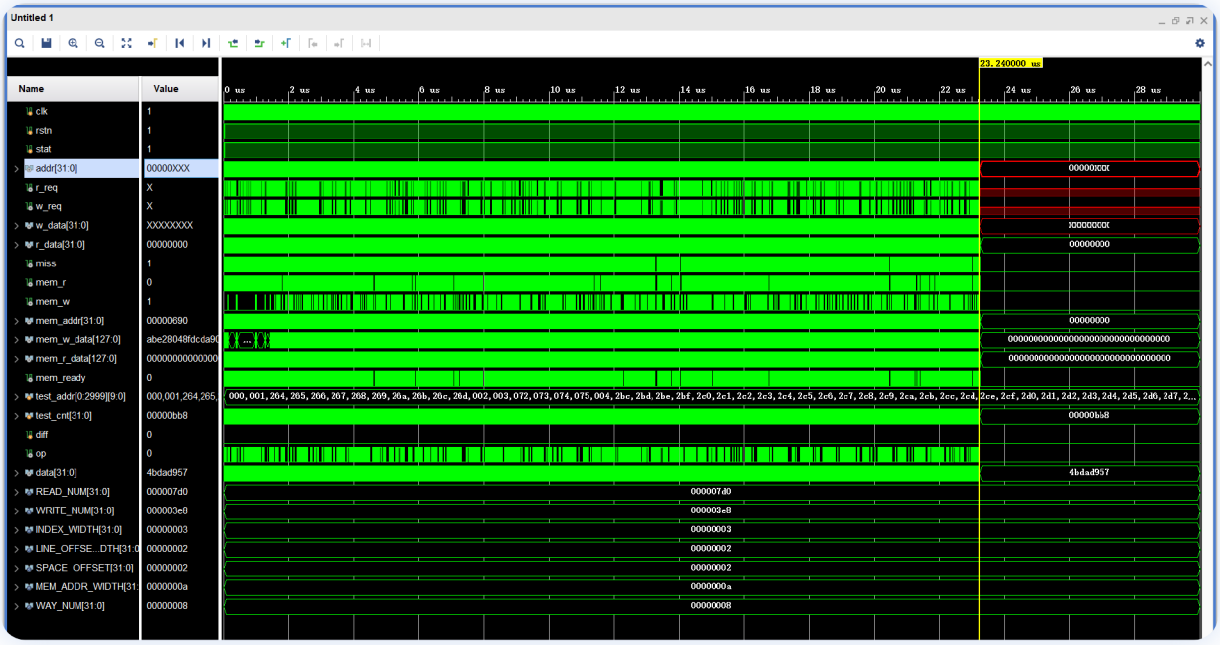
### 二路



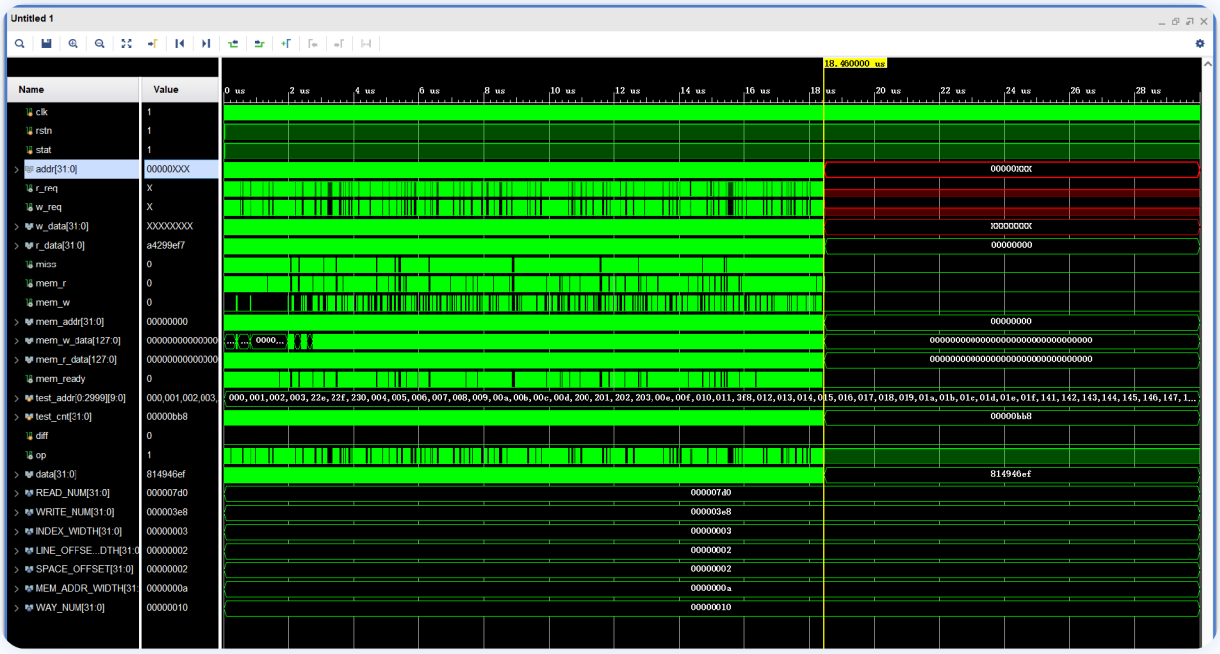

### 四路




### 八路




## 十六路



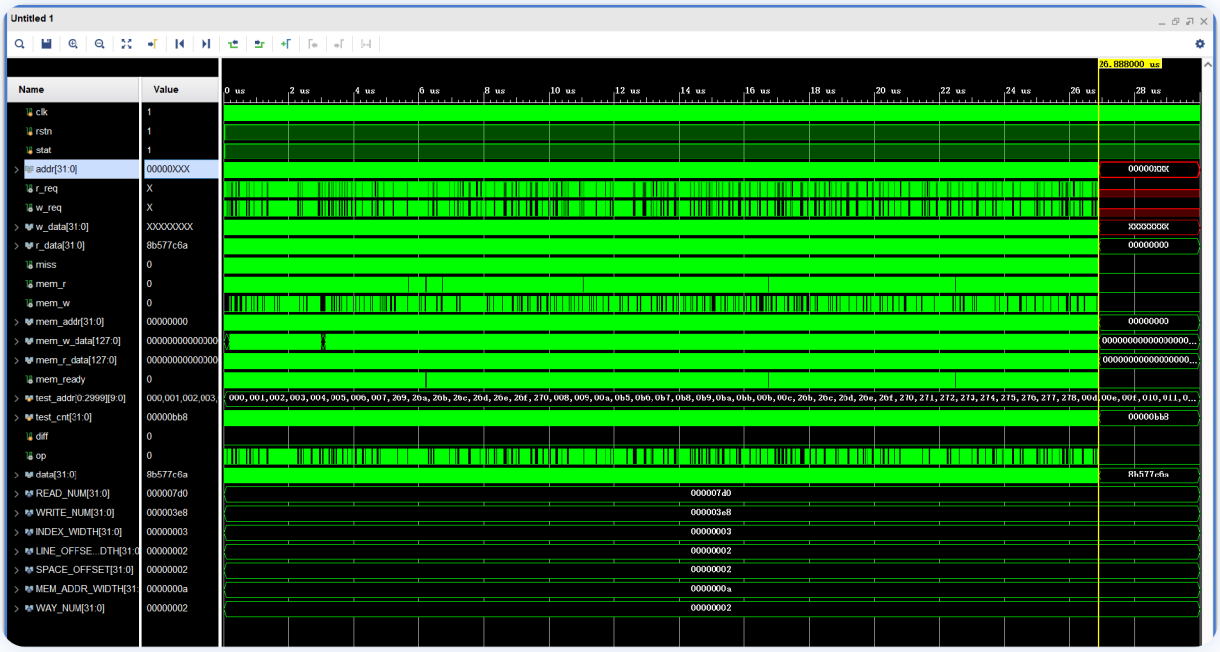

# 任务 2：多样化的替换策略

## LRU 替换策略

设置一个二维数组，第一个索引值对应 `index`，第二个索引值对应 `Cache set` 中 `Cache line` 的序号。在每次访问 `Cache line` 时，将该次所访问对应的 `age` 清零，并增加其他 `age` 的数值。最后在遍历时选择 `age` 最大者用以替换即可，此即近期最少使用者。

## FIFO 替换策略

实现逻辑与 LRU 类似，使用 `age` 记录 `Cache line` 被替换进入的总时长，每个时钟周期均会增加 `age` 的数值。选择 `age` 最大的 `Cache line` 用以替换即可，也即最早被替换进入者，同时将其 `age` 清零。与 LRU 的区别在于，先进先出不考虑最近是否被访问，只关心被替换的先后顺序。




## 随机替换策略

设置一个 `counter` 变量，在每个时钟周期加一，并设置其在 `0 ~ WAY_NUM-1` 之间循环，当需要替换时，则选择此时的 `counter` 作为对应的 `block`，用于选择 `cache line`。

### 分析不同替换策略的优劣:

仿真测试结果中，不同策略累计时间相差不大，其中 LRU 略优于 FIFO 及随机替换策略。

- LRU
  - 替换掉最久未使用的缓存块通常更合理，能有效降低缓存未命中率。
  - 每次访问缓存时都需要更新数据结构，可能带来额外的时间开销，尤其是在缓存块数量较多时，时间可能耗费较多。
- FIFO
  - 只需要一个时间戳来记录缓存块按进入缓存的顺序进行替换。实现时只需在每次缓存满时替换最早进入缓存的块。
  - 可能导致一些频繁访问的块被不恰当地替换，从而增加了缓存未命中率。
- 随机替换
  - 实现起来也非常简单，只需在缓存满时随机选择一个缓存块进行替换。
  - 由于是随机选择，性能可能会显得不太稳定，存在一定的不确定性。